

1. Initial State

1	2	3
	4	6
7	5	8

RIGHT

1	2	3
4		6
7	5	8

I

DOWN

1	2	3
7	4	6
	5	8

II

UP

	2	3
1	4	6
7	5	8

III

D. MANIKANDAN
715524243026
ARTIFICIAL INTELLIGENCE
IIIrd Year - BTECH A&DS

I)

1	2	3
4		5
7	5	8

DOWN

1	2	3
4	5	6
7		8

RIGHT

1	2	3
4	5	6
7	8	

GOAL STATE

II)

1	2	3
7	4	6
	5	8

RIGHT

1	2	3
7	4	6
5		8

UP

1	2	3
7		6
5	4	8

LEFT

1	2	3
	7	6
5	4	8

DOWN

1	2	3
	5	6
4	7	8

LEFT

1	2	3
5		6
4	7	8

UP

1	2	3
5	7	6
4		8

RIGHT

1	2	3
5	7	6
	4	8

DOWN

1	2	3
4	5	6
	7	8

RIGHT

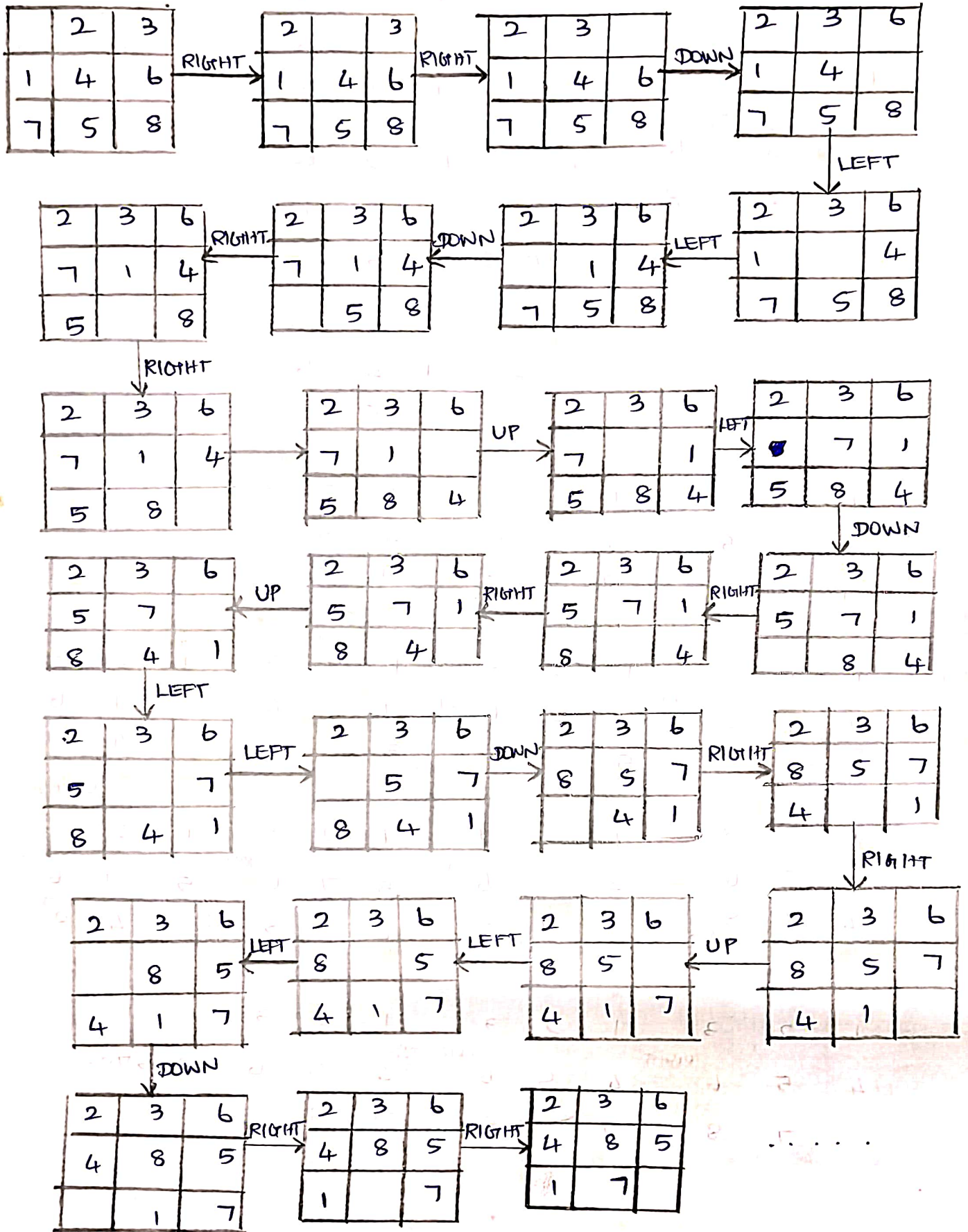
1	2	3
4	5	6
7		8

RIGHT

1	2	3
4	5	6
7	8	

GOAL STATE

(ii)



PROGRAM:

```
def dis_Ventose(initial):  
    stack = [(initial, [])]  
    visited = set()  
    step-count = 0  
    while stack:  
        state, path = stack.pop()  
        state-tuple = board.to-tuple(state)  
        if state-tuple in visited:  
            continue  
        visited.add(state-tuple)  
        step-count += 1  
        print(f"step {step-count}: Exploring state with path {path}")  
        print_board(state)  
        if state == GOAL-STATE:  
            print("Goal Reached")  
            print(f"Total steps taken: {len(path)}")  
            print("Moves:")  
            for i in path:  
                print(i)  
            return path  
        for move, neighbour in reversed(get_neighbours(state)):  
            if board.to-tuple(neighbour) not in visited:  
                stack.append((neighbour, path+[move]))  
                print()  
    print("No solution found")  
    return None
```

Total of 404 steps as told by program:

from copy import deepcopy

GOAL-STATE = [[1,2,3],[4,5,6],[7,8,0]]

DIRECTIONS = [("UP", -1, 0), ("DOWN", 1, 0), ("LEFT", 0, -1), ("RIGHT", 0, 1)]

def board_to_tuple(board):

return tuple(tuple(row) for row in board)

def print_board(board):

for j in board:

print(" ".join(str(num) if num != 0 else " " for i in c))

print()

def find_blank(board):

for i in range(3):

for j in range(3):

if board[i][j] == 0:

return i, j

def get_neighbours(board):

neighbours = []

x, y = find_blank(board)

for move, dx, dy in DIRECTIONS:

ny, nx = y + dy, x + dx

if 0 <= nx < 3 and 0 <= ny < 3:

new_board = deepcopy(board)

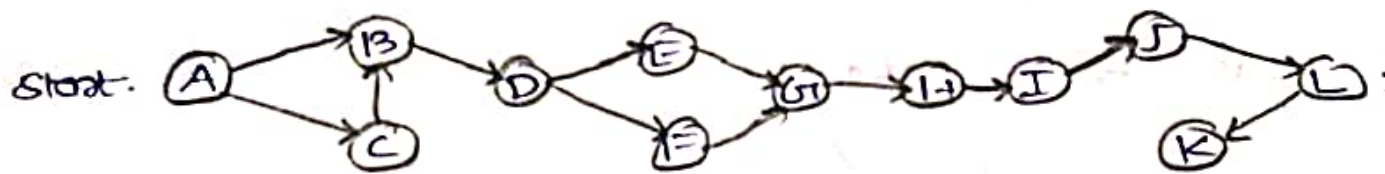
new_board[x][y], new_board[nx][ny] =

new_board[nx][ny], new_board[x][y]

neighbours.append((move, new_board))

return neighbours

2) BI-DIRECTIONAL SEARCH:



Using BFS:

Queue Node	Visited
A	
B C	A
C D	B
D E F	C
EF	D
F G	E
G	F
H	G
I	H
J	I
L	J
	L

Intersection at : L

Path: A-B-C-D-E-F-G-H-I-J-L

Path cost : 10

Using DFS.

Visited	Stack
	A
A	B
B	D
D	E
E	G
G	H
H	I
I	J
J	(L)
(L)	L

Insertion : L

Path : A-B-D-E-G-H-I-J-L

Path Cost : 8